Each group only needs to have one member to upload the document and the GitHub link. The document includes

1. **Fully dressed use cases**
    - At least four functional requirements (i.e., uses cases) for each team member
    - Note: There are multiple ways to describe functional requirements. Please refer to the format of fully dressed use cases mentioned in our lecture on "Requirements" to develop each use case. Specifically, each use case should have the following format:

Use Case UC1: Process Sale

Primary Actor: Cashier
Stakeholders and interests:
E.g., Cashier: want accurate and fast payment
Preconditions
Success guarantee
Main success scenario
Extensions
Special requirements
Technology and data variation list

Each use case should have an ID and a title to summarize the requirement (e.g., Process Sale). Each field in the form shown above must exist in the form, although some fields can be blank. For instance, the field of "Special requirements" can be blank, if there is no case-specific non-functional requirement to mention.

2. **Use case diagrams**

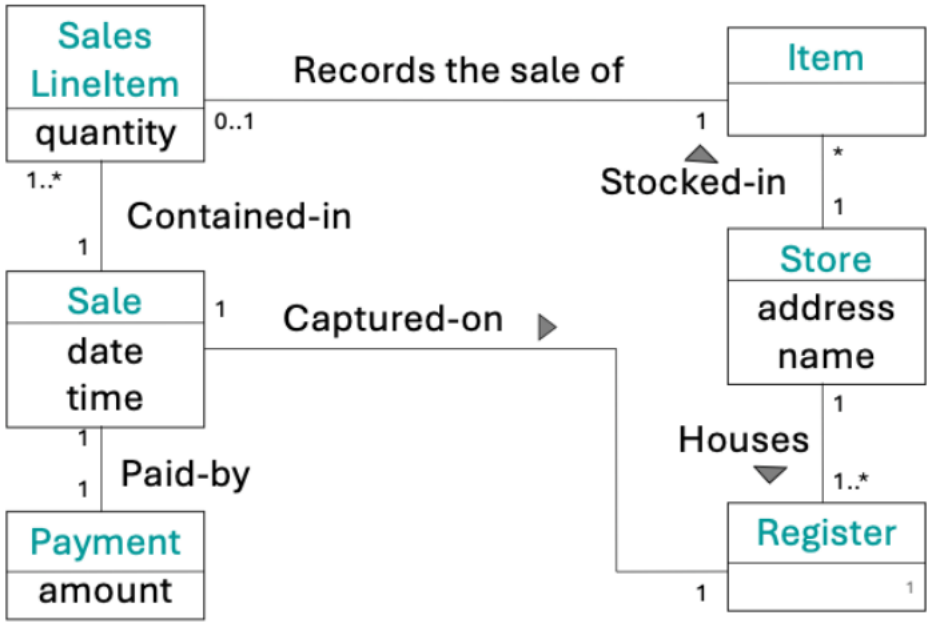
If all use case notations can be put into a single diagram, creating and presenting one diagram should be good enough. However, when the use cases do not quite fit into one readable diagram, please split that diagram to smaller ones, and draw them one-by-one.

3. **Conceptual class diagrams**

Similar to what is mentioned above, if all classes can fit into one diagram, presenting one diagram should be good enough. However, when there are many classes that do not quite fit into one readable diagram, please split the diagram into smaller class diagrams.

Please draw one or more class diagrams to visualize the conceptual classes, their fields/attributes, and relationship between classes. DON'T include methods/functions, access modifiers for fields (e.g., public, private, or protected), or data types of fields. When specifying relationship between classes, just visualize "Generalization" and "Association" relationship. When the "Association"

edge is used, please add a direction mark, a brief explanation to describe the relationship, and the numeric multiplicity. For instance, the diagram shown below is reasonable.



4. Supplementary specifications

- At least one non-functional requirement for each team member

Grading rubrics: Points can be deducted if

1. This formatting guideline is not followed,
2. Some of the described requirements do not quite make sense or violate common sense. For instance, if the multiplicity relationship between Sale and SalesLineItem is marked as 1-to-1, then the numeric relationship is wrong as each sale can contain multiple sales line items.

For more information, please refer to all lecture notes related to “Requirements”.

Additional clarification:

1. GitHub will store all documents and software artifacts you create. You can encourage all team members to leverage GitHub to coordinate on their documentation development, back up their own progress, or even resolve conflicts when multiple team members edit the same text line. Additionally, please ensure that your team use the feature "Issues" to resolve some issue or have discussions on certain part of the project. The usage of "Issues" is also counted towards your final score.

2. **Supplementary specifications** is outside all use case descriptions. As mentioned on Slide 10 of SE3-Requirement-1-concepts.pdf, Requirements Analysis basically leads to two major artifacts: Use Cases and Supplementary Specification. The uses cases focus on functional requirements, and supplementary specification focuses on non-functional requirements. As our lecture focuses on the development of use case description and there is little to expand on non-functional requirements, I didn't put further format specification on non-functional requirements. It is OK that you just provide some simple explanation for each of the non-functional requirements you have in the section "Supplementary specifications".